```
"Combatant.h"
---------
#pragma once

#include <Urho3D/Scene/Component.h>
#include <Urho3D/Graphics/AnimationController.h>
#include <Urho3D/Scene/Node.h>
#include <Urho3D/Core/StringUtils.h>

using namespace Urho3D;

enum SIDE { SIDE_HERO = 0, SIDE_VILLAIN = 1 };

class Combatant : public Component
{
    URHO3D_OBJECT(Combatant, Component);

public:
    Combatant(Context* context)
        : Component(context)
        , node_(nullptr)
        , animController_(nullptr)
        , isFrozen_(false)
        , freezeTimer_(0.0f)
        , side_(0)
        , maxHealth_(100)
        , health_(100)
    {}

    virtual void Initialize(Node* node, const ea::string& prefabPath) = 0;
    virtual void Update(float timeStep) = 0;
    virtual bool CheckCollision(Combatant* other) = 0;
    virtual void HandleHit(Combatant* attacker) = 0;

    // Make SetFrozen VIRTUAL so Zombie can override it
    virtual void SetFrozen(bool freeze, bool lockAnimation = true) = 0;

    virtual void Reset() = 0;

    // Common utility accessors
    virtual float GetForwardOffset() const { return 0.0f; } // Added for lunge tracking

    Node* GetNode() const { return node_; }
    Vector3 GetPosition() const { return node_ ? node_->GetWorldPosition() : Vector3::ZERO; }
    void SetPosition(const Vector3& pos) { if (node_) node_->SetWorldPosition(pos); }

    bool IsFrozen() const { return isFrozen_; }

protected:
    Node* node_;
    AnimationController* animController_;
    bool isFrozen_;
    float freezeTimer_;
    int side_;
    int maxHealth_;
    int health_;
};
"CombatSystem.cpp"
---------
#include "CombatSystem.h"
#include <Urho3D/IO/Log.h>
#include <Urho3D/Resource/ResourceCache.h>
#include <Urho3D/Scene/Scene.h>
#include <Urho3D/Graphics/AnimationController.h>
#include <Urho3D/Physics/RigidBody.h>
#include <cmath>

CombatSystem::CombatSystem(Context* context)
    : Component(context)
    , fighter_(nullptr)
    , zombie_(nullptr)
    , debugText_(nullptr)
    , arenaHitFrozen_(false)
{
    // HARDCODED PRODUCTION METRICS CONVERTED DIRECTLY FROM YOUR SYSTEM TWEAK LOGS
    zombieOffsetScale_ = 0.820f;
    zombieOffsetRotation_ = 78.0f;       // Fixed structural 90-degree asset rotation layout bug
    fighterOffsetRotation_ = 0.0f;
    zombieOffsetPosition_ = Vector3(-0.530f, 0.0f, 0.000f); // Calibrated gap boundary offset
    fighterOffsetPosition_ = Vector3(0.000f, 0.0f, 0.000f);
}

void CombatSystem::Initialize(Fighter* fighter, Zombie* zombie, Text* uiText)
{
    fighter_ = fighter;
    zombie_ = zombie;
    debugText_ = uiText;
    arenaHitFrozen_ = false;
}

void CombatSystem::UpdateCombat(float timeStep)
{
    if (!fighter_ || !zombie_) return;

    // 1. Regular approach update logic
    if (!arenaHitFrozen_ && !zombie_->IsNeckBiting())
    {
        zombie_->CheckCollision(fighter_);
    }

    // 2. Weapon slice confirmation trace
    if (fighter_->IsMidSwing() && !zombie_->IsFrozen() && zombie_->IsContactFrozen() && !arenaHitFrozen_)
    {
        float deltaX = std::abs(fighter_->GetPosition().x_ - zombie_->GetPosition().x_);
        if (deltaX < 1.4f)
        {
            arenaHitFrozen_ = true;
            zombie_->HandleHit(fighter_);
            if (debugText_)
            {
                debugText_->SetText("CHOP DETECTED! PRESS SPACEBAR TO SPAWN NEXT RUNNER.");
                debugText_->SetColor(Color::RED);
            }
            return;
        }
    }

    // 3. Perfect Sync Bite Sequence - Structural Transition Launch
    if (!arenaHitFrozen_ && zombie_->IsContactFrozen() && !zombie_->IsNeckBiting() && !zombie_->IsFrozen() && !fighter_->IsMidSwing())
    {
        static float biteDelayTimer = 0.0f;
        biteDelayTimer += timeStep;
        if (biteDelayTimer > 0.40f)
        {
            biteDelayTimer = 0.0f;
            arenaHitFrozen_ = true;

            zombie_->StartNeckBite(fighter_->GetNode());
            zombie_->SetBiteSynced(true);
            fighter_->SetBiteSynced(true);

            if (debugText_)
            {
                debugText_->SetText("NECK BITE SEQUENCING ACTIVE.");
                debugText_->SetColor(Color::YELLOW);
            }

            Node* fNode = fighter_->GetNode();
            Node* zNode = zombie_->GetNode();

            if (fNode && zNode)
            {
                // KEEP DYNAMIC COLLIDERS ACTIVE: Enforce capsule separation walls safely
                if (fighter_->GetRigidBody()) {
                    fighter_->GetRigidBody()->SetKinematic(false);
                    fighter_->GetRigidBody()->SetLinearVelocity(Vector3::ZERO);
                    fighter_->GetRigidBody()->SetAngularVelocity(Vector3::ZERO);
                }
                if (zombie_->GetRigidBody()) {
                    zombie_->GetRigidBody()->SetKinematic(false);
                    zombie_->GetRigidBody()->SetLinearVelocity(Vector3::ZERO);
                    zombie_->GetRigidBody()->SetAngularVelocity(Vector3::ZERO);
                }

                // Apply production orientation coordinates
                fNode->SetRotation(Quaternion(0.0f, fighterOffsetRotation_, 0.0f));
                zNode->SetRotation(Quaternion(0.0f, zombieOffsetRotation_, 0.0f));

                Vector3 currentFighterPos = fNode->GetPosition();
                currentFighterPos.y_ = 0.0f;
                fNode->SetPosition(currentFighterPos);

                Vector3 targetZombiePos = currentFighterPos + zombieOffsetPosition_;
                zNode->SetPosition(targetZombiePos);

                if (fighter_->GetRigidBody()) fighter_->GetRigidBody()->SetPosition(currentFighterPos);
                if (zombie_->GetRigidBody()) zombie_->GetRigidBody()->SetPosition(targetZombiePos);

                // Run synchronized track branches recursively via children lookups
                AnimationController* fAnim = nullptr;
                ea::vector<Node*> fChildren;
                fNode->GetChildrenWithComponent<AnimationController>(fChildren, true);
                if (auto* topF = fNode->GetComponent<AnimationController>()) fAnim = topF;
                else if (!fChildren.empty() && fChildren.front()) fAnim = fChildren.front()->GetComponent<AnimationController>();

                if (fAnim)
                {
                    ea::string path = "fighter-nek-bite/test.glb.d/Animations/Armature.001_Armature.001_mixamo.com_Layer0.ani";
                    fAnim->StopAll();
                    AnimationParameters params(context_, path);
                    params.looped_ = true;
                    fAnim->PlayNewExclusive(params);
                }

                AnimationController* zAnim = nullptr;
                ea::vector<Node*> zChildren;
                zNode->GetChildrenWithComponent<AnimationController>(zChildren, true);
                if (auto* topZ = zNode->GetComponent<AnimationController>()) zAnim = topZ;
                else if (!zChildren.empty() && zChildren.front()) zAnim = zChildren.front()->GetComponent<AnimationController>();

                if (zAnim)
                {
                    ea::string path = "zombie-nek-bite/test.glb.d/Animations/Armature_Armature_mixamo.com_Layer0_0.ani";
                    zAnim->StopAll();
                    AnimationParameters params(context_, path);
                    params.looped_ = true;
                    zAnim->PlayNewExclusive(params);
                }
            }
        }
    }

    // ========================================================================
    // PERMANENT SPATIAL MERGE ENFORCEMENT FRAME CONTROL
    // ========================================================================
    if (arenaHitFrozen_ && zombie_->IsNeckBiting())
    {
        Node* fNode = fighter_->GetNode();
        Node* zNode = zombie_->GetNode();

        if (fNode && zNode)
        {
            // Lock orientations perfectly face-to-face on every single frame loop update
            fNode->SetRotation(Quaternion(0.0f, fighterOffsetRotation_, 0.0f));
            zNode->SetRotation(Quaternion(0.0f, zombieOffsetRotation_, 0.0f));

            // Snap fighter to floor baseline coordinates
            Vector3 fPos = fNode->GetPosition();
            fPos.y_ = 0.0f;
            fNode->SetPosition(fPos);

            // FORCE ENFORCED MERGE SEPARATION: Explicit vector lock blocks body merging completely
            Vector3 zPos = fPos + zombieOffsetPosition_;
            zNode->SetPosition(zPos);

            // Hard sync the physical hulls to stop gravity drops or sliding collisions
            if (fighter_->GetRigidBody()) {
                fighter_->GetRigidBody()->SetPosition(fPos);
                fighter_->GetRigidBody()->SetLinearVelocity(Vector3::ZERO);
                fighter_->GetRigidBody()->SetAngularVelocity(Vector3::ZERO);
            }
            if (zombie_->GetRigidBody()) {
                zombie_->GetRigidBody()->SetPosition(zPos);
                zombie_->GetRigidBody()->SetLinearVelocity(Vector3::ZERO);
                zombie_->GetRigidBody()->SetAngularVelocity(Vector3::ZERO);
            }
        }
    }
}

void CombatSystem::HandleManualAttack()
{
    if (!fighter_ || !zombie_) return;

    if (zombie_->IsFrozen() || zombie_->IsNeckBiting())
    {
        URHO3D_LOGINFO("[SYSTEM-INPUT] SPACEBAR: Resetting arena stage...");
        ResetSimulation();
        return;
    }

    URHO3D_LOGINFO("[SYSTEM-INPUT] SPACEBAR: Executing fighter axe swing command.");
    fighter_->StartAttack();
}

void CombatSystem::ResetSimulation()
{
    arenaHitFrozen_ = false;

    if (fighter_) fighter_->Reset();
    if (zombie_) zombie_->Reset();

    if (debugText_)
    {
        debugText_->SetText("STAGE ACTIVE: RUNNING...");
        debugText_->SetColor(Color::GREEN);
    }
}

"CombatSystem.h"
---------
#pragma once
#include <Urho3D/Scene/Component.h>
#include <Urho3D/UI/Text.h>
#include "Fighter.h"
#include "Zombie.h"

using namespace Urho3D;

class CombatSystem : public Component
{
    URHO3D_OBJECT(CombatSystem, Component);

public:
    CombatSystem(Context* context);
    ~CombatSystem() override = default;

    void Initialize(Fighter* fighter, Zombie* zombie, Text* uiText);
    void UpdateCombat(float timeStep);
    void HandleManualAttack();
    void ResetSimulation();

    bool IsArenaHitFrozen() const { return arenaHitFrozen_; }

private:
    Fighter* fighter_;
    Zombie* zombie_;
    Text* debugText_;
    bool arenaHitFrozen_;

    float zombieOffsetScale_;
    float zombieOffsetRotation_;
    float fighterOffsetRotation_;
    Vector3 zombieOffsetPosition_;
    Vector3 fighterOffsetPosition_;
};

"Fighter.cpp"
---------
#include "Fighter.h"
#include <Urho3D/IO/Log.h>
#include <Urho3D/Core/Context.h>
#include <Urho3D/Resource/ResourceCache.h>
#include <Urho3D/Scene/PrefabResource.h>
#include <Urho3D/Graphics/AnimatedModel.h>
#include <cstring>

// Physics collision layers
enum CollisionLayer {
    LAYER_WORLD = 1,
    LAYER_FIGHTER = 2,
    LAYER_ZOMBIE = 4
};

Fighter::Fighter(Context* context) : LogicComponent(context)
{
    SetUpdateEventMask(USE_FIXEDUPDATE);
}

void Fighter::RegisterObject(Context* context)
{
    context->RegisterFactory<Fighter>();
}

void Fighter::Initialize(Node* node, const ea::string& prefabPath)
{
    node_ = node;
    idleAnimPath_.clear();
    attackAnimPath_ = "test.glb.d/Animations/Armature_Armature_mixamo.com_Layer0.ani";

    if (node_)
    {
        arenaPosition_ = node_->GetPosition();
        auto* cache = GetSubsystem<ResourceCache>();
        if (cache && !prefabPath.empty())
        {
            auto* prefabFile = cache->GetResource<PrefabResource>(prefabPath);
            if (prefabFile)
            {
                meshNode_ = node_->CreateChild("AxeManMesh");
                prefabFile->InstantiateReference(meshNode_);
                meshNode_->SetRotation(Quaternion(0.0f, -90.0f, 0.0f));
                meshNode_->SetScale(5.0f / 9.0f);

                Vector3 pos = meshNode_->GetPosition();
                pos.y_ = 0.05f;
                meshNode_->SetPosition(pos);

                animController_ = meshNode_->FindComponent<AnimationController>();
                if (animController_) animController_->StopAll();
            }
        }

        if (meshNode_) ReparentWeapon(meshNode_);
        SetupPhysics();
    }
    Reset();
}

void Fighter::Initialize(Node* node, const ea::string& idleAnimPath, const ea::string& attackAnimPath)
{
    node_ = node;
    idleAnimPath_ = idleAnimPath;
    attackAnimPath_ = attackAnimPath;
    if (node_)
    {
        arenaPosition_ = node_->GetPosition();
        animController_ = node_->GetOrCreateComponent<AnimationController>();
        SetupPhysics();
    }
    Reset();
}

void Fighter::SetupPhysics()
{
    if (!node_) return;
    rigidBody_ = node_->GetOrCreateComponent<RigidBody>();
    if (rigidBody_)
    {
        rigidBody_->SetMass(80.0f);
        rigidBody_->SetUseGravity(true); // Gravity active like Ninja Sample
        rigidBody_->SetLinearDamping(10.0f);
        rigidBody_->SetAngularDamping(10.0f);
        rigidBody_->SetKinematic(false);
     rigidBody_->SetCollisionLayer(LAYER_FIGHTER);
     rigidBody_->SetCollisionMask(LAYER_WORLD | LAYER_ZOMBIE); // Collide only with world and zombie

        // NINJA SAMPLE RULE: Freeze all angular rotations (X, Y, Z rotation tipping over)
        // This forces the capsule to stay perfectly vertical on the ground grid at all times
        rigidBody_->SetAngularFactor(Vector3::ZERO);
    }
    collisionShape_ = node_->GetOrCreateComponent<CollisionShape>();
    if (collisionShape_) collisionShape_->SetCapsule(0.45f, 1.6f, Vector3(0.0f, 0.8f, 0.0f));
}

void Fighter::Reset()
{
    isSwinging_ = false;
    isBiteSynced_ = false;
    swingTimer_ = 0.0f;

    if (node_)
    {
        node_->SetPosition(arenaPosition_);
        node_->SetRotation(Quaternion::IDENTITY);
        if (rigidBody_)
        {
            rigidBody_->SetKinematic(false);
            rigidBody_->SetAngularFactor(Vector3::ZERO); // Re-enforce lock
            rigidBody_->SetPosition(arenaPosition_);
            rigidBody_->SetRotation(Quaternion::IDENTITY);
            rigidBody_->SetLinearVelocity(Vector3::ZERO);
            rigidBody_->ResetForces();
            rigidBody_->Activate();
        }
    }
    ResetMeshLocalTransform();
    if (animController_) animController_->StopAll();
}

void Fighter::ResetMeshLocalTransform()
{
    if (meshNode_)
    {
        meshNode_->SetRotation(Quaternion(0.0f, -90.0f, 0.0f));
        meshNode_->SetScale(5.0f / 9.0f);
        meshNode_->SetPosition(Vector3(0.0f, 0.05f, 0.0f));
    }
}

void Fighter::StartAttack()
{
    if (isSwinging_ || isBiteSynced_) return;
    if (!animController_) return;

    isSwinging_ = true;
    swingTimer_ = 0.0f;
    animController_->StopAll();

    AnimationParameters params(context_, attackAnimPath_);
    params.looped_ = false;
    animController_->PlayNewExclusive(params);
    animController_->SetTime(attackAnimPath_, 0.0f);
    animController_->SetSpeed(attackAnimPath_, 1.0f);
}

void Fighter::FixedUpdate(float timeStep)
{
    if (isBiteSynced_) return;

    if (isSwinging_)
    {
        swingTimer_ += timeStep;
        if (!animController_->IsPlaying(attackAnimPath_))
        {
            isSwinging_ = false;
            swingTimer_ = 0.0f;
            ResetMeshLocalTransform();

            if (rigidBody_)
            {
                rigidBody_->SetPosition(arenaPosition_);
                rigidBody_->SetLinearVelocity(Vector3::ZERO);
                rigidBody_->Activate();
            }
        }
    }

    if (rigidBody_ && !isSwinging_)
    {
        Vector3 pos = rigidBody_->GetPosition();
        Vector3 drift = arenaPosition_ - pos;
        drift.y_ = 0.0f;
        if (drift.Length() > 0.001f)
        {
            rigidBody_->SetPosition(arenaPosition_);
            rigidBody_->SetLinearVelocity(Vector3::ZERO);
        }
        if (node_->GetPosition() != arenaPosition_) node_->SetPosition(arenaPosition_);
    }
}

void Fighter::ReparentWeapon(Node* meshNode)
{
    if (!meshNode) return;
    auto* animatedModel = meshNode->FindComponent<AnimatedModel>();
    if (!animatedModel) return;

    Skeleton& skeleton = animatedModel->GetSkeleton();
    Bone* rightHandBone = skeleton.GetBone("RightHand");
    if (rightHandBone && rightHandBone->node_)
    {
        ea::vector<Node*> allDescendants;
        ea::vector<Node*> stack = { meshNode };
        while (!stack.empty())
        {
            Node* current = stack.back();
            stack.pop_back();
            for (Node* child : current->GetChildren())
            {
                allDescendants.push_back(child);
                stack.push_back(child);
            }
        }
        for (Node* child : allDescendants)
        {
            auto* sm = child->GetComponent<StaticModel>();
            if (!sm) continue;
            Model* mdl = sm->GetModel();
            if (!mdl || std::strstr(mdl->GetName().c_str(), "BattleAxe") == nullptr) continue;

            Vector3 worldPos = child->GetWorldPosition();
            Quaternion worldRot = child->GetWorldRotation();
            child->SetParent(rightHandBone->node_);
            Quaternion handInv = rightHandBone->node_->GetWorldRotation().Inverse();
            child->SetPosition(handInv * (worldPos - rightHandBone->node_->GetWorldPosition()));
            child->SetRotation(handInv * worldRot);
        }
    }
}

"Fighter.h"
---------
#pragma once
#include <Urho3D/Scene/LogicComponent.h>
#include <Urho3D/Graphics/AnimationController.h>
#include <Urho3D/Physics/RigidBody.h>
#include <Urho3D/Physics/CollisionShape.h>
#include <Urho3D/Graphics/AnimatedModel.h>

using namespace Urho3D;

class Fighter : public LogicComponent
{
    URHO3D_OBJECT(Fighter, LogicComponent);

public:
    Fighter(Context* context);
    ~Fighter() override = default;

    static void RegisterObject(Context* context);

    void Initialize(Node* node, const ea::string& prefabPath);
    void Initialize(Node* node, const ea::string& idleAnimPath, const ea::string& attackAnimPath);
    void Reset();
    void StartAttack();
    void FixedUpdate(float timeStep) override;

    void SetBiteSynced(bool synced) { isBiteSynced_ = synced; }
    bool IsSwinging() const { return isSwinging_; }
    bool IsMidSwing() const { return isSwinging_ && (swingTimer_ >= 0.20f && swingTimer_ <= 0.60f); }
    Vector3 GetPosition() const { return node_ ? node_->GetPosition() : Vector3::ZERO; }
    Quaternion GetRotation() const { return node_ ? node_->GetRotation() : Quaternion::IDENTITY; }
    RigidBody* GetRigidBody() const { return rigidBody_; }

private:
    void ReparentWeapon(Node* meshNode);
    void ResetMeshLocalTransform();
    void SetupPhysics();
    void SetupBoneColliders();

    SharedPtr<Node> node_;
    SharedPtr<Node> meshNode_;
    SharedPtr<AnimationController> animController_;
    SharedPtr<RigidBody> rigidBody_;
    SharedPtr<CollisionShape> collisionShape_;

    ea::string idleAnimPath_;
    ea::string attackAnimPath_;

    bool isSwinging_{false};
    bool isBiteSynced_{false};
    float swingTimer_{0.0f};
    Vector3 arenaPosition_{Vector3::ZERO};
};

"main.cpp"
---------
#include <Urho3D/Urho3D.h>
#include <Urho3D/Engine/Engine.h>
#include <Urho3D/Engine/EngineDefs.h>
#include <Urho3D/Graphics/Camera.h>
#include <Urho3D/Graphics/Light.h>
#include <Urho3D/Graphics/Octree.h>
#include <Urho3D/Graphics/Renderer.h>
#include <Urho3D/Graphics/Viewport.h>
#include <Urho3D/Graphics/Zone.h>
#include <Urho3D/Graphics/DebugRenderer.h>
#include <Urho3D/Graphics/StaticModel.h>
#include <Urho3D/Graphics/Model.h>
#include <Urho3D/Graphics/Material.h>
#include <Urho3D/Container/Str.h>
#include <Urho3D/IO/Log.h>
#include <Urho3D/Resource/ResourceCache.h>
#include <Urho3D/Scene/Scene.h>
#include <Urho3D/Scene/PrefabResource.h>
#include <Urho3D/Input/Input.h>
#include <Urho3D/Graphics/GraphicsDefs.h>
#include <Urho3D/UI/Text.h>
#include <Urho3D/UI/UI.h>
#include <Urho3D/UI/Font.h>
#include <Urho3D/Core/Timer.h>
#include <Urho3D/Physics/PhysicsWorld.h>

// Include our new object-oriented components
#include "Fighter.h"
#include "Zombie.h"
#include "CombatSystem.h"

using namespace Urho3D;

int main(int argc, char** argv)
{
    SharedPtr<Context> context(new Context());
    SharedPtr<Engine> engine(new Engine(context));

    StringVariantMap engineParameters;
    engineParameters[EP_WINDOW_TITLE] = "Kilo-Code Battle Arena";
    engineParameters[EP_RESOURCE_PREFIX_PATHS] = "Data";
    engineParameters[EP_FULL_SCREEN] = false;
    engineParameters[EP_WINDOW_WIDTH] = 1152;
    engineParameters[EP_WINDOW_HEIGHT] = 864;
    engineParameters[EP_MULTI_SAMPLE] = 1;
    engineParameters[EP_VSYNC] = true;

    if (!engine->Initialize(engineParameters, {}))
    {
        URHO3D_LOGERROR("Could not initialise the engine.");
        return 1;
    }

    // REGISTER CUSTOM COMPONENT FACTORIES SO THE ENGINE CAN REFLECT THEM
    context->RegisterFactory<Fighter>();
    context->RegisterFactory<Zombie>();
    context->RegisterFactory<CombatSystem>();

    auto* renderer = engine->GetSubsystem<Renderer>();
    auto* cache    = engine->GetSubsystem<ResourceCache>();
    auto* input    = engine->GetSubsystem<Input>();
    auto* ui       = engine->GetSubsystem<UI>();
    auto* time     = engine->GetSubsystem<Time>();

    // ---- Diagnostics HUD Overlay ----
    auto* font = cache->GetResource<Font>("Fonts/Anonymous Pro.ttf");
    auto* debugText = ui->GetRoot()->CreateChild<Text>();
    if (font && debugText)
    {
        debugText->SetText("SPACEBAR: ATTACK / RESPAWN | W/S: ZOOM | A/D: ORBIT");
        debugText->SetFont(font, 16);
        debugText->SetColor(Color::GREEN);
        debugText->SetHorizontalAlignment(HA_CENTER);
        debugText->SetVerticalAlignment(VA_TOP);
    }

     // ---- World Scene Layout Initialization ----
     SharedPtr<Scene> scene(new Scene(context));
     scene->CreateComponent<Octree>();
     scene->CreateComponent<DebugRenderer>();
     scene->CreateComponent<PhysicsWorld>(); // Required for RigidBody components

    auto* z = scene->CreateComponent<Zone>();
    z->SetBoundingBox(BoundingBox(-1000.0f, 1000.0f));
    z->SetAmbientColor(Color(0.35f, 0.35f, 0.35f));
    z->SetFogColor(Color(0.55f, 0.55f, 0.55f));
    z->SetFogStart(150.0f);
    z->SetFogEnd(250.0f);

    Node* l = scene->CreateChild("KeyLight");
    l->SetDirection(Vector3(0.2f, -0.8f, 0.5f));
    auto* dl = l->CreateComponent<Light>();
    dl->SetLightType(LIGHT_DIRECTIONAL);
    dl->SetColor(Color(0.8f, 0.784f, 0.76f));
    dl->SetBrightness(0.72f);
    dl->SetCastShadows(false);

    Node* fillLightNode = scene->CreateChild("FillLight");
    fillLightNode->SetDirection(Vector3(-0.4f, -0.6f, -0.2f));
    auto* fillLight = fillLightNode->CreateComponent<Light>();
    fillLight->SetLightType(LIGHT_DIRECTIONAL);
    fillLight->SetColor(Color(0.4f, 0.4f, 0.44f));
    fillLight->SetBrightness(0.32f);

    auto* cameraNode = scene->CreateChild("Camera");
    cameraNode->SetPosition(Vector3(0.0f, 1.2f, -3.5f));
    cameraNode->LookAt(Vector3(0.0f, 0.6f, 1.6f));
    auto* camera = cameraNode->CreateComponent<Camera>();

    Node* camLightNode = scene->CreateChild("ViewerLight");
    camLightNode->SetPosition(cameraNode->GetPosition());
    camLightNode->SetRotation(cameraNode->GetRotation());
    auto* camLight = camLightNode->CreateComponent<Light>();
    camLight->SetLightType(LIGHT_SPOT);
    camLight->SetColor(Color(1.2f, 1.12f, 1.04f));
    camLight->SetBrightness(3.2f);
    camLight->SetRange(40.0f);
    camLight->SetFov(72.0f);
    camLight->SetCastShadows(false);

    SharedPtr<Viewport> viewport(new Viewport(context, scene, camera));
    viewport->SetDrawDebug(true);
    renderer->SetViewport(0, viewport);

    // ========================================================================
    // COMPONENT CREATION & BINDING POOL
    // ========================================================================

    // 1. Create and attach the Fighter component
    Node* axeManPivot = scene->CreateChild("AxeManPivot");
    axeManPivot->SetPosition(Vector3(0.59f, 0.0f, 1.6f));
    auto* fighterComponent = axeManPivot->CreateComponent<Fighter>();
    fighterComponent->Initialize(axeManPivot, "test.glb.d/Prefab.prefab");

    // 2. Create and attach the Zombie component
    Node* zombiePivot = scene->CreateChild("ZombiePivot");
    zombiePivot->SetPosition(Vector3(-2.0f, 0.0f, 1.6f));  // Moved closer to be visible on screen
    auto* zombieComponent = zombiePivot->CreateComponent<Zombie>();
    const ea::string zombiePrefabPath = "firing-running-didn'twork/test_out/test.gltf.d/Prefab.prefab";
    zombieComponent->Initialize(zombiePivot, zombiePrefabPath);

    // 3. Setup the Master Combat System Arbitrator Component
    Node* coreSystemNode = scene->CreateChild("CombatSystemNode");
    auto* combatSystem = coreSystemNode->CreateComponent<CombatSystem>();
    combatSystem->Initialize(fighterComponent, zombieComponent, debugText);

    // ========================================================================
    // MAIN LOOP
    // ========================================================================
      while (!engine->IsExiting())
      {
          engine->RunFrame();
          float dt = time->GetTimeStep();

          // --- ZOMBIE AUTOPSY: run every frame, but only log every 30th to save IO ---
          static int debugFrame = 0;
          debugFrame++;
          bool logNow = (debugFrame % 30 == 1);

          if (logNow && zombieComponent)
          {
              Node* zPivot = zombieComponent->GetNode();
              if (zPivot)
              {
                  URHO3D_LOGINFOF("[Z-AUTOPSY] Pivot    pos=%s rot=%s",
                                  zPivot->GetPosition().ToString().c_str(),
                                  zPivot->GetRotation().ToString().c_str());

                  Node* zMesh = zPivot->GetChild("ZombieMesh");
                  if (zMesh)
                  {
                      URHO3D_LOGINFOF("[Z-AUTOPSY] Mesh     worldPos=%s scale=%s",
                                      zMesh->GetWorldPosition().ToString().c_str(),
                                      zMesh->GetScale().ToString().c_str());

                      AnimatedModel* am = zMesh->FindComponent<AnimatedModel>();
                      if (!am) am = zPivot->FindComponent<AnimatedModel>(); // fallback
                      if (am)
                      {
                          BoundingBox bb = am->GetWorldBoundingBox();
                          URHO3D_LOGINFOF("[Z-AUTOPSY] WorldBB  min=%s max=%s size=%s",
                                          bb.min_.ToString().c_str(),
                                          bb.max_.ToString().c_str(),
                                          bb.Size().ToString().c_str());
                          URHO3D_LOGINFOF("[Z-AUTOPSY] Drawable enabled=%d castShadows=%d",
                                          am->IsEnabled(), am->GetCastShadows());
                      }
                      else
                      {
                          URHO3D_LOGWARNING("[Z-AUTOPSY] NO AnimatedModel found!");
                      }
                  }
                  else
                  {
                      URHO3D_LOGWARNING("[Z-AUTOPSY] NO 'ZombieMesh' child found!");
                  }

                  RigidBody* zBody = zombieComponent->GetRigidBody();
                  if (zBody)
                  {
                      URHO3D_LOGINFOF("[Z-AUTOPSY] Physics  pos=%s vel=%s mass=%.1f",
                                      zBody->GetPosition().ToString().c_str(),
                                      zBody->GetLinearVelocity().ToString().c_str(),
                                      zBody->GetMass());
                  }
              }
          }

          // --- VISUAL DEBUG DRAW ---
          auto* dbg = scene->GetComponent<DebugRenderer>();
          if (dbg)
          {
              dbg->SetEnabled(true); // make sure it's on

              if (zombieComponent && zombieComponent->GetNode())
              {
                  Node* zPivot = zombieComponent->GetNode();
                  // Red wire sphere at pivot
                  dbg->AddSphere(Sphere(zPivot->GetWorldPosition(), 0.2f), Color::RED);

                  Node* zMesh = zPivot->GetChild("ZombieMesh");
                  if (zMesh)
                  {
                      AnimatedModel* am = zMesh->FindComponent<AnimatedModel>();
                      if (am)
                      {
                          // Green wire bounding box of the actual model
                          dbg->AddBoundingBox(am->GetWorldBoundingBox(), Color::GREEN);
                      }
                  }
              }

              // Also draw fighter for reference
              if (fighterComponent && fighterComponent->GetNode())
              {
                  Node* fPivot = fighterComponent->GetNode();
                  dbg->AddSphere(Sphere(fPivot->GetWorldPosition(), 0.2f), Color::BLUE);
                  AnimatedModel* fam = fPivot->FindComponent<AnimatedModel>();
                  if (!fam && fPivot->GetChild("AxeManMesh"))
                      fam = fPivot->GetChild("AxeManMesh")->FindComponent<AnimatedModel>();
                  if (fam)
                      dbg->AddBoundingBox(fam->GetWorldBoundingBox(), Color::CYAN);
              }
          }

          if (input->GetKeyPress(KEY_SPACE))
         {
             if (combatSystem->IsArenaHitFrozen())
             {
                 combatSystem->ResetSimulation();
             }
             else
             {
                 combatSystem->HandleManualAttack();
             }
         }

         combatSystem->UpdateCombat(dt);

        // Camera controls
        const Vector3 screenCenterAnchor(0.0f, 0.6f, 1.6f);
        Vector3 anchorToCamVector = cameraNode->GetPosition() - screenCenterAnchor;

        bool orbitLeft  = input->GetKeyDown(KEY_A) || input->GetKeyDown(KEY_Q);
        bool orbitRight = input->GetKeyDown(KEY_D) || input->GetKeyDown(KEY_E);

        if (orbitLeft || orbitRight)
        {
            float rotationDirection = orbitLeft ? -90.0f : 90.0f;
            rotationDirection *= dt;
            Quaternion yawOrbitRotation(rotationDirection, Vector3::UP);
            anchorToCamVector = yawOrbitRotation * anchorToCamVector;
            cameraNode->SetPosition(screenCenterAnchor + anchorToCamVector);
        }

        if (input->GetKeyDown(KEY_W)) cameraNode->Translate(Vector3::FORWARD * 3.0f * dt);
        if (input->GetKeyDown(KEY_S)) cameraNode->Translate(Vector3::BACK * 3.0f * dt);

        cameraNode->LookAt(screenCenterAnchor);

        camLightNode->SetPosition(cameraNode->GetPosition());
        camLightNode->SetRotation(cameraNode->GetRotation());

         if (input->GetKeyPress(KEY_ESCAPE)) engine->Exit();

     }

    renderer->SetViewport(0, nullptr);
    engine->Exit();

    return 0;
}
"Zombie.cpp"
---------
#include "Zombie.h"
#include "Fighter.h"
#include <Urho3D/IO/Log.h>
#include <Urho3D/Core/Context.h>
#include <Urho3D/Resource/ResourceCache.h>
#include <Urho3D/Scene/PrefabResource.h>
#include <cmath>

Zombie::Zombie(Context* context) : LogicComponent(context)
{
    SetUpdateEventMask(USE_FIXEDUPDATE);
}

void Zombie::RegisterObject(Context* context)
{
    context->RegisterFactory<Zombie>();
}

void Zombie::Initialize(Node* node, const ea::string& prefabPath)
{
    node_ = node;
    if (!node_) return;

    startPosition_ = node_->GetPosition();
    node_->SetRotation(Quaternion(0.0f, 90.0f, 0.0f));

    size_t lastSlash = prefabPath.find_last_of('/');
    runAnimPath_ = (lastSlash != ea::string::npos) ? prefabPath.substr(0, lastSlash) + "/Animations/mixamo.com.ani" : prefabPath;

    meshNode_ = node_->CreateChild("ZombieMesh");
    auto* cache = GetSubsystem<ResourceCache>();
    auto* prefab = cache->GetResource<PrefabResource>(prefabPath);
    if (prefab)
    {
        prefab->InstantiateReference(meshNode_);
        animController_ = meshNode_->FindComponent<AnimationController>();
        meshNode_->SetScale(75.0f);
        meshNode_->SetPosition(Vector3::ZERO);
        meshNode_->SetRotation(Quaternion::IDENTITY);
        SetupPhysics();
    }
    Reset();
}

void Zombie::SetupPhysics()
{
    if (!node_) return;
    rigidBody_ = node_->GetOrCreateComponent<RigidBody>();
    if (rigidBody_)
    {
        rigidBody_->SetMass(60.0f);
        rigidBody_->SetUseGravity(false);
        rigidBody_->SetLinearDamping(5.0f);
        rigidBody_->SetAngularDamping(10.0f);
        rigidBody_->SetKinematic(false); // Default active movement
        rigidBody_->SetCollisionLayer(4);
        rigidBody_->SetCollisionMask(0xFFFFFFFF);
    }
    collisionShape_ = node_->GetOrCreateComponent<CollisionShape>();
    if (collisionShape_) collisionShape_->SetCapsule(0.45f, 1.6f, Vector3(0.0f, 0.8f, 0.0f)); // Match fighter capsule thickness
}

void Zombie::FixedUpdate(float timeStep)
{
    if (isBiteSynced_ || health_ <= 0 || isFrozen_) return;

    if (!isContactFrozen_)
    {
        Vector3 pos = node_->GetPosition();
        pos.x_ += runSpeed_ * timeStep;
        node_->SetPosition(pos);

        if (rigidBody_)
        {
            rigidBody_->SetPosition(pos);
            rigidBody_->SetLinearVelocity(Vector3::ZERO);
        }
    }
    else if (rigidBody_)
    {
        rigidBody_->SetLinearVelocity(Vector3::ZERO);
    }
}

void Zombie::CheckCollision(Fighter* fighter)
{
    if (!node_ || !fighter || isFrozen_ || health_ <= 0 || isNeckBiting_ || isBiteSynced_) return;

    Vector3 myPos = node_->GetPosition();
    Vector3 targetPos = fighter->GetPosition();
    float boundaryX = targetPos.x_ - solidStopThreshold_;
    float deltaX = myPos.x_ - targetPos.x_;

    if (isContactFrozen_)
    {
        if (myPos.x_ > boundaryX)
        {
            node_->SetPosition(Vector3(boundaryX, myPos.y_, myPos.z_));
            if (rigidBody_)
            {
                rigidBody_->SetPosition(Vector3(boundaryX, myPos.y_, myPos.z_));
                rigidBody_->SetLinearVelocity(Vector3::ZERO);
            }
        }
        return;
    }

    if (myPos.x_ >= boundaryX)
    {
        node_->SetPosition(Vector3(boundaryX, myPos.y_, myPos.z_));
        if (rigidBody_)
        {
            rigidBody_->SetPosition(Vector3(boundaryX, myPos.y_, myPos.z_));
            rigidBody_->SetLinearVelocity(Vector3::ZERO);
        }
        // FIXED: Appended the correct member underscore variable suffix here
        isContactFrozen_ = true;
    }
}

void Zombie::HandleHit(Fighter* fighter)
{
    if (hasBeenHit_) return;
    hasBeenHit_ = true;
    isFrozen_ = true;
    if (animController_) animController_->StopAll();
    if (rigidBody_)
    {
        rigidBody_->SetLinearVelocity(Vector3::ZERO);
        rigidBody_->SetAngularVelocity(Vector3::ZERO);
    }
}

void Zombie::ApplyBoundaryFreeze()
{
    if (isContactFrozen_) return;
    isContactFrozen_ = true;
    if (rigidBody_) rigidBody_->SetLinearVelocity(Vector3::ZERO);
}

void Zombie::TriggerNeckBite()
{
    if (isNeckBiting_) return;
    isNeckBiting_ = true;
    if (rigidBody_) rigidBody_->SetLinearVelocity(Vector3::ZERO);
}

void Zombie::StartNeckBite(Node* targetNode)
{
    TriggerNeckBite();
}

void Zombie::Reset()
{
    isFrozen_ = false;
    isContactFrozen_ = false;
    isNeckBiting_ = false;
    isBiteSynced_ = false;
    hasBeenHit_ = false;
    health_ = maxHealth_;

    if (node_)
    {
        node_->SetPosition(startPosition_);
        node_->SetRotation(Quaternion(0.0f, 90.0f, 0.0f));
        if (rigidBody_)
        {
            rigidBody_->SetKinematic(false); // Reset to standard dynamics
            rigidBody_->SetPosition(startPosition_);
            node_->SetRotation(Quaternion(0.0f, 90.0f, 0.0f));
            rigidBody_->SetLinearVelocity(Vector3::ZERO);
            rigidBody_->ResetForces();
            rigidBody_->Activate();
        }
    }

    if (animController_ && !runAnimPath_.empty())
    {
        animController_->StopAll();
        AnimationParameters params(context_, runAnimPath_);
        params.looped_ = true;
        animController_->PlayNewExclusive(params);
        animController_->SetTime(runAnimPath_, 0.0f);
        animController_->SetSpeed(runAnimPath_, 0.5f);
    }
}
"Zombie.h"
---------
#pragma once
#include <Urho3D/Scene/LogicComponent.h>
#include <Urho3D/Graphics/AnimationController.h>
#include <Urho3D/Physics/RigidBody.h>
#include <Urho3D/Physics/CollisionShape.h>
#include <Urho3D/Graphics/AnimatedModel.h>

using namespace Urho3D;

class Fighter;

class Zombie : public LogicComponent
{
    URHO3D_OBJECT(Zombie, LogicComponent);

public:
    Zombie(Context* context);
    ~Zombie() override = default;

    static void RegisterObject(Context* context);

    void Initialize(Node* node, const ea::string& prefabPath);
    void Reset();
    void FixedUpdate(float timeStep) override;

    void TriggerNeckBite();
    void StartNeckBite(Node* targetNode);
    void ApplyBoundaryFreeze();
    void CheckCollision(Fighter* fighter);
    void HandleHit(Fighter* fighter);

    void SetBiteSynced(bool synced) { isBiteSynced_ = synced; }
    bool IsFrozen() const { return isFrozen_; }
    bool IsContactFrozen() const { return isContactFrozen_; }
    bool IsNeckBiting() const { return isNeckBiting_; }
    Vector3 GetPosition() const { return node_ ? node_->GetPosition() : Vector3::ZERO; }
    RigidBody* GetRigidBody() const { return rigidBody_; }

private:
    void SetupPhysics();

    SharedPtr<Node> node_;
    SharedPtr<Node> meshNode_;
    SharedPtr<AnimationController> animController_;
    SharedPtr<RigidBody> rigidBody_;
    SharedPtr<CollisionShape> collisionShape_;

    ea::string runAnimPath_;
    Vector3 startPosition_{-4.5f, 0.0f, 1.6f};

    bool isFrozen_{false};
    bool isContactFrozen_{false};
    bool isNeckBiting_{false};
    bool isBiteSynced_{false};

    float health_{100.0f};
    const float maxHealth_{100.0f};
    const float runSpeed_{0.90f};
    const float solidStopThreshold_{1.2f};
    bool hasBeenHit_ = false;
};
```